

SD Basic - System and Environment

Asking SD about itself, about the machine, and about the session you are in.

This page covers the enquiries: what time is it, who am I, where is the data, what did the last thing that failed say, and what may this account do. Most of it is one function, `system()`, and most of the surprises are about which form a path comes back in.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Values that are particular to one machine — a user number, a computer name, a process id — are marked as examples; the shapes are not. What you see back will differ in the value and match in the form.

SYSTEM()

```
system(key)
```

THE SESSION

KEY		VALUE
7	terminal type	windows
9	CPU time used, ms	45
12	time, as <code>time()</code>	70787
18	user number	67 (example)
23	break key enabled?	1
24	echo enabled?	1
25	is this a phantom?	0
26	prompt character	?
1000	<code>capturing</code> in effect?	0

KEY		VALUE
1001	case inversion on?	0
1029	internal subroutine depth	0 at the top, 1 inside a gosub
1030	login time, internal	1851017987
1031	operating system process id	605 (example)
1050	administrator?	0

THE MACHINE

KEY		VALUE
31	licence number	0
42	IP address	empty
91	is this Windows?	1
1006	Windows NT style?	0
1009	endian — 0 little	0
1010	platform name	Windows
1012	SD version	W1.0-0
1013 / 1014	user limit, without / with the phantom pool	20 / 20
1015	computer name	Gitorli (example)
1017	port number of a tcp connection	0
1028	system id	1028

Two of those three answer correctly and one does not. `system(91)` reads 1 and `system(1010)` reads Windows. `system(1006)`, "Windows NT style?", reads 0 — it is the one to leave alone. Ask `system(91)` **whether this is Windows**; it is the key this port sets deliberately for that purpose.

PATHS, AND THEY ARE NOT ALL IN THE SAME FORM

KEY		VALUE
32	the sdsys directory	C:\ProgramData\SD\sdsys

KEY		VALUE
38	the temporary directory	/cygdrive/c/WINDOWS/TEMP
1011	the configuration file	C:/ProgramData/SD/sd.conf
1024	the directory SD was started in	/cygdrive/c/Users/dmont/OneDrive/Documents

Three different spellings of a Windows PATH come out of one function. A backslash path, a POSIX `/cygdrive/` path, and a forward-slash path with a drive letter. `@sdsys` agrees with key 32 and `@path` — the account directory — is in the POSIX form.

A `/cygdrive/` path handed to a Windows program does not work. Windows reads it as drive-relative and either fails silently or complains that the parent directory does not exist. This is not theoretical: it is what stopped the full-screen editors working the first time they were built for this port. There is a conversion function in the kernel and **an ordinary program cannot call it** — see "What is not here". **Take the path from configuration rather than from `system()` if a Windows program is going to see it.**

LISTS AND STRUCTURES

KEY		
1002	the call stack	field per level: <code>path</code> , then <code>offset</code> and line pairs — <code>.../BP.OUT/ZZMATH</code> at line 41, then <code>\$CPROC</code>
1003	open files	field per file, <code>unit</code> and path. <code>\$ipc</code> is always one of them
1025	environment variables	two fields: field 1 every name, field 2 every value, value-mark separated

`system(1025)` returns **2** fields, every name in the first — it is not a list of `NAME=value` pairs.

TIME

KEY		
1005	internal time	<code>date() * 86400 + time()</code> — the difference is 0
1020	milliseconds since midnight	9587454

`system(1020)` is the one to time something with: against a lock wait it gives 252 ms where `time()` gives 0.

DATE, TIME and TIMEDATE

date() time() timedate()

Taken together at one instant:

date()	21423 — days since 31 December 1967
time()	70787 — seconds since midnight
timedate()	19:39:47 26 AUG 2026
oconv(date(), 'D4-')	08-26-2026
oconv(time(), 'MTS')	19:39:47

@date and @time hold the same numbers, **but they are set once per command** rather than read afresh, so in a long loop they do not move while date() and time() do.

timedate() returns a formatted string, not a number. Do not do arithmetic on it — see SD Basic - Data Conversion for the conversion codes.

ENV()

env(name)

env() is case sensitive and a wrong case looks exactly like a missing variable.

env('PATH')	926 characters
env('path')	0 characters
env('ProgramData')	C:\ProgramData
env('NOSUCHVAR')	empty

Windows itself treats environment variable names as case-insensitive everywhere else, which is what makes this worth knowing. `ProgramData` works only because that is exactly how Windows spells it. **Get the spelling from `system(1025)` field 1 rather than from memory.**

CONFIG()

```
config(name)
```

On a stock installation:

<code>config('FILERULE')</code>	<code>0</code>
<code>config('GRPSIZE')</code>	<code>2</code>
<code>config('MAXIDLEN')</code>	<code>63</code>
<code>config('NUMFILES')</code>	<code>80</code>
<code>config('NUMLOCKS')</code>	<code>100</code>
<code>config('SORTMEM')</code>	<code>4096</code>
<code>config('SPOOLER')</code>	<code>empty</code>

The name is case sensitive and at most eight characters. both failures now look the same, which is the point — a name that is too long is a name that does not exist, and a caller cannot tell the two apart:

<code>config('numlocks')</code> — right name, wrong case	<code>empty, status() 1004</code>
<code>config('NOSUCHKEY')</code> — nine characters	<code>empty, status() 1004</code>

Keep every `config()` name to eight characters and upper case.

Neither call aborts the caller: a name that is too long comes back empty with a status, the same as a name that does not exist.

SYSMSG()

```
sysmsg(number {, substitution ...})
```

Returns the text of one of SD's own messages, with %s substitutions filled in.

sysmsg(2831)	Unrecognised statement
--------------	------------------------

sysmsg(6711, 'ABC')	Unable to find source record ABC
---------------------	----------------------------------

sysmsg(2201)	Account name '' is not in register — an unfilled substitution comes back empty
--------------	--

sysmsg(99999)	[99999] Message not found
---------------	---------------------------

sysmsg(1)	[1] Message not found
-----------	-----------------------

A message number is not a status() code. They are separate numbering schemes that overlap. status() 3006 is *record not found*; sysmsg(3006) is Modes: . Do not render a status code by passing it to sysmsg().

get.messages() takes **no arguments** and returned **nothing** — zero fields — in an ordinary session. It reports messages sent between sessions, and nothing had sent any.

Continued in

SD Basic - Status, Encryption and the Machine — status codes, checksums, encryption, umask, the @variables, os.execute and logmsg.